



PONTIFICIA UNIVERSIDAD CATOLICA DE CHILE  
ESCUELA DE INGENIERIA  
DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

**Criptografía y Seguridad Computacional - IIC3253**  
**Rúbrica Tarea 2**

## Preguntas

1. En esta pregunta usted va a implementar un sistema de cifrado simétrico basado en la red de Feistel. Al igual que en el caso de AES, este sistema de cifrado se define para bloques de un cierto largo, y luego se utilizan modos de operación que indican cómo se cifra un mensaje de largo arbitrario usando el cifrado de bloques. Adicionalmente, y al igual que para AES, se debe utilizar una función de padding para que el largo del mensaje a cifrar sea divisible por el largo del bloque.

### Definición del sistema de cifrado

Considere un largo de bloque  $2n$  (el largo de un bloque debe ser par), suponga dada una función de hash  $h : \{0, 1\}^* \rightarrow \{0, 1\}^n$ , y fije un parámetro  $r \geq 1$  como el número de rondas de la red de Feistel. Dados estos parámetros, una red de Feistel define un sistema de cifrado simétrico donde  $\mathcal{M} = \mathcal{C} = \{0, 1\}^{2n}$  y  $\mathcal{K} = \{0, 1\}^*$  (vale decir, se puede usar una clave de largo arbitrario). Dado un mensaje  $m \in \{0, 1\}^{2n}$  y una clave  $k \in \{0, 1\}^*$ , la función de cifrado  $Enc_k : \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n}$  para un bloque se define de la siguiente forma. El mensaje se divide como  $m = m_1 \| m_2$ , donde  $m_1 \in \{0, 1\}^n$  y  $m_2 \in \{0, 1\}^n$ , y se define la siguiente secuencia  $\{L_i, R_i\}_{i \in \{0, \dots, r\}}$  de strings:  $L_0 = m_1$ ,  $R_0 = m_2$ , y para cada  $i \in \{0, \dots, r-1\}$ :

$$\begin{aligned} L_{i+1} &= R_i, \\ R_{i+1} &= L_i \oplus h(k_{i+1} \| R_i), \end{aligned}$$

donde  $k_{i+1} = h(k \| i+1)$  (esto es un key schedule, como el que es usado en AES). Finalmente, se tiene que  $Enc_k(m) = L_r \| R_r$ .

### La clase Feistel a implementar

En esta pregunta usted debe implementar en Python una clase `Feistel` que define un sistema de cifrado simétrico basado en la red de Feistel. Esta clase debe tener los siguientes componentes:

- Un atributo `block_length` que almacena el largo del bloque, el cual debe ser un número par.
- Un atributo `number_rounds` que almacena el número de rondas de la red de Feistel, el cual debe ser mayor o igual que 1.
- Un atributo `hash_function` que almacena la función de hash que se usará en la red de Feistel.
- Un atributo `secret_key` que corresponde a la clave secreta que se usará en el cifrado y descifrado de mensajes.
- Un constructor que recibe el largo del bloque, el número de rondas, la función hash que se utilizará en la red de Feistel, y la clave secreta.
- Una función `enc_block` que cifra un mensaje `m` de largo `block_length` utilizando la clave `secret_key`. `enc_block` implementa la función  $Enc_{secret\_key}$  descrita anteriormente, y no realiza padding para calcular el cifrado.
- Una función `dec_block` que descifra un mensaje `c` de largo `block_length` utilizando la clave `secret_key`. `dec_block` implementa la inversa de la función  $Enc_{secret\_key}$  descrita anteriormente.
- Una función `key_schedule` que genera la clave para la ronda `i` de la red de Feistel. Para implementar esta función use la instrucción `str(i).encode()` para transformar `i` en una secuencia de bytes.
- Una función `pad` que agrega a un mensaje `m` un símbolo 1 seguido de un número arbitrario de símbolos 0 hasta que el largo del string generado es divisible por `block_length`. Si el largo de `m` ya es divisible por `block_length`, entonces se agrega un nuevo bloque a `m` de la forma `10...0` de largo `block_length`.
- Una función `unpad` que elimina los símbolos extra agregados a un mensaje `m` mediante la función `pad`.
- Una función `enc` que recibe como parámetros un mensaje `m` de largo arbitrario y un modo de operación `mode`. Esta función cifra el resultado de aplicar la función `pad` a `m` usando la clave `secret_key` y considerando el modo de operación `mode`. Los dos modos de operación posibles son `ECB` y `CBC`, los cuales son definidos como en el caso de AES. Si el modo de operación es `ECB`, entonces `enc` retorna el mensaje cifrado. Si el modo de operación es `CBC`, entonces `enc` genera de manera aleatoria un vector de inicialización `IV` que utiliza para cifrar, y retorna una tupla con el mensaje cifrado y el vector de inicialización `IV`.
- Una función `dec` que recibe como parámetros un mensaje `c` de largo arbitrario, un modo de operación `mode`, y un parámetro opcional `IV` que representa un vector de inicialización. Esta función descifra el mensaje `c` usando la clave `secret_key` y considerando el modo de operación `mode`, y luego utiliza la función `unpad` para eliminar los símbolos extra agregados al cifrar un mensaje. Los dos modos de operación posibles son `ECB` y `CBC`. Ambos modos son definidos como en el caso de AES, y para el caso de `CBC` es necesario incluir el vector de inicialización `IV` como parámetro.

Todos los mensajes y claves en su implementación deben ser de tipo `bytes`. En particular, cuando se indica que un bloque tiene largo 32, significa que el bloque está formado por

32 bytes, lo cual equivale a 256 bits. Todas las operaciones deben realizarse sobre bytes. Por ejemplo, la operación  $a \oplus b$  se debe aplicar sobre dos bytes  $a$  y  $b$ , lo cual en Python es equivalente a transformar  $a$  y  $b$  en sus representaciones binarias  $a'$  y  $b'$ , respectivamente, realizar  $a' \oplus b'$  bit a bit para obtener un resultado  $c$  en binario, y finalmente transformar  $c$  a un byte. Además, debe utilizar **raise** para generar excepciones cuando hay errores en los parámetros utilizados; por ejemplo, si el largo del bloque no es un número par, si el número de rondas es igual a 0 o si se intenta descifrar un mensaje usando CBC sin un vector de inicialización IV. Finalmente, se debe utilizar la librería **typing** para especificar los tipos de los parámetros y los valores retornados por las funciones.

## Detalles de la implementación de la clase Feistel

En esta tarea usted debe completar la siguiente implementación de la clase `Feistel`, donde además puede agregar las funciones auxiliares que estime conveniente.

```
class Feistel:
    def __init__(
        self,
        block_length: int,
        number_rounds: int,
        hash_function: Callable[[bytes], bytes],
        secret_key: bytes,
    ) -> None:
        pass

    def enc_block(self, m: bytes) -> bytes:
        pass

    def dec_block(self, c: bytes) -> bytes:
        pass

    def key_schedule(self, i: int) -> bytes:
        pass

    def pad(self, m: bytes) -> bytes:
        pass

    def unpad(self, m: bytes) -> bytes:
        pass

    def enc(self, m: bytes, mode: str) -> bytes | Tuple[bytes, bytes]:
        pass

    def dec(self, c: bytes, mode: str, IV: Optional[bytes] = None) -> bytes:
        pass
```

## Verificación de la correctitud de la implementación

Para probar si su implementación es correcta se va a utilizar la siguiente implementación de la función de hash SHA256:

```

from hashlib import sha256

def SHA256(message: bytes) -> bytes:
    alg = sha256()
    alg.update(message)
    return alg.digest()

```

Considerando esta función, usted puede verificar si su implementación funciona correctamente utilizando pruebas como la siguiente, donde se especifica que los bloques son de 64 bytes (puesto que SHA256 produce un hash con 32 bytes), y se indica que la red de Feistel tiene 10 rondas:

```

net = Feistel(64, 10, SHA256, b'T.d66w7ydubyw3h-Hsy')

m1 = b'1234567890123456789012345678901234567890123456789012345678901234'
c1 = net.enc_block(m1)
d1 = net.dec_block(c1)

m2 = b'Este es un mensaje de largo arbitrario'
c2 = net.enc(m2, 'ECB')
d2 = net.dec(c2, 'ECB')

m3 = b'Este es un mensaje de largo arbitrario'
(c3, iv) = net.enc(m3, 'CBC')
d3 = net.dec(c3, 'CBC', iv)

print(f'Texto plano      : {m1}')
print(f'Texto cifrado    : {c1}')
print(f'Texto descifrado  : {d1}\n')
print(f'Texto plano      : {m2}')
print(f'Texto cifrado     : {c2}')
print(f'Texto descifrado  : {d2}\n')
print(f'Texto plano      : {m3}')
print(f'Texto cifrado     : {c3}')
print(f'Texto IV          : {iv}')
print(f'Texto descifrado  : {d3}')

```

El resultado de utilizar este código debería ser el siguiente:

```

Texto plano      : b'1234567890123456789012345678901234567890123456789012345678901234'
Texto cifrado    : b'0\xa6\x9c)\xe4\xf9\x04\xbb\xb7\xebS\x98^\x1c5\x1at1\xd7\xc8\xde
                  \xf4\xac\x8e\x08\x95/\x07\x0f\xac\xc2\xf5\xba\x89\xf3Z\xe1\x12\xf5
                  \x10o\xea\xe2!\xe0\xfe\xfd++\x01\xd7\xb8C\xa9\xc9$\xcb\xbfG\xfdj\xa8uq'
Texto descifrado : b'1234567890123456789012345678901234567890123456789012345678901234'

Texto plano      : b'Este es un mensaje de largo arbitrario'
Texto cifrado    : b'\xf7\x80.~\x01\xb7\xf2\xc4\xd6\xb1I\x125\xae\xe3\xb9\x8dq\xbcP\x14
                  \xf7\x84\xaf\x94\x83\xa9%\x17\xa2?\xeet\xc9=\xb0L+\x08\x04q\x14J\x06
                  \\x18\xbc\xa9t\xdc\x87\xe0\xff\x0a0s\xb7\x17\xca\x08\xd0_\xde\xdb'
Texto descifrado : b'Este es un mensaje de largo arbitrario'

Texto plano      : b'Este es un mensaje de largo arbitrario'
Texto cifrado    : b'V.K\x04\x92\x84\x16yS\xccI\xf1\x01\xaaX\x1a\x8fW{\x8e=HaJ\x85\x12$\xa8

```

```

      \x93;X\x05\x08\x0c\xaa8\x82Az\x14/l'\x10\xdf\t{\xcfk\xcd\xba\ng\xed6
      \xe\xeb\xa8,\x15\xe3\x9e\xdb@'
IV      : b'K\xe5\x7fng\x9cJ\x8c\xbez\x05k\xe0\x1e\xe2\x19\x92\xe3u\x05\x88/#\x04
      !G(\x16\xf8w\x06\x96\x94tS*|+\x16e6\xf0\x1c\x96\x86\xdbZp\xe1\xf3(\xdf
      \xb6iE[\xfa\x1f\xdd6\xae\x07\x0'
Texto descifrado : b'Este es un mensaje de largo arbitrario'

```

**Corrección.** La corrección de la tarea va a estar basado en un conjuntos de tests. El primero de éstos se basa en el uso de 64 bytes como largo del bloque (`block.length`), 10 rondas, y la función de hash `SHA256`. En este caso se va a fijar una clave secreta, y se van a realizar los siguientes tests:

- [0.15 puntos] Con un mensaje cuyo largo es divisible por `block.length` se debe obtener el texto cifrado correcto al utilizar `enc_block`.
- [0.15 puntos] Con un mensaje cuyo largo no es divisible por `block.length` se debe obtener una excepción al utilizar `enc_block`.
- [0.15 puntos] Con un texto cifrado cuyo largo es divisible por `block.length` se debe obtener el mensaje descifrado correcto al utilizar `dec_block`.
- [0.15 puntos] Con un texto cifrado cuyo largo no es divisible por `block.length` se debe obtener una excepción al utilizar `dec_block`.
- [0.15 puntos] Con un número de rondas dado se debe obtener la clave correcta al utilizar `key_schedule`.
- [0.15 puntos] Con un mensaje cuyo largo no es divisible por `block.length` se debe obtener el padding correcto al utilizar `pad`.
- [0.15 puntos] Con un mensaje cuyo largo es divisible por `block.length` se debe obtener el padding correcto al utilizar `pad`.
- [0.15 puntos] Con un padding correctamente generado utilizando `pad`, se debe obtener el mensaje original al utilizar `unpad`.
- [0.15 puntos] Con un padding cuyo largo no es divisible por `block.length` se debe obtener una excepción al utilizar `unpad`.
- [0.15 puntos] Con un padding cuyo largo es divisible por `block.length` pero que no sigue las reglas de padding definidas en la tarea, se debe obtener una excepción al utilizar `unpad`.
- [0.15 puntos] Con un mensaje cuyo largo es divisible por `block.length` y en modo de operación `ECB`, se debe obtener el cifrado correcto al utilizar `enc`.
- [0.15 puntos] Con un mensaje cuyo largo no es divisible por `block.length` y en modo de operación `ECB`, se debe obtener el cifrado correcto al utilizar `enc`.
- [0.15 puntos] Con un texto cifrado cuyo largo es divisible por `block.length` y en modo de operación `ECB`, se debe obtener el descifrado correcto al utilizar `dec`.
- [0.15 puntos] Con un texto cifrado cuyo largo no es divisible por `block.length` y en modo de operación `ECB`, se debe obtener una excepción al utilizar `dec`.

- [0.45 puntos] Con un mensaje cuyo largo es divisible por `block_length` y en modo de operación `CBC`, se debe obtener un texto cifrado del largo correcto y un vector de inicialización `IV` al utilizar `enc`. Al utilizar `dec` con el texto cifrado y el vector de inicialización `IV` se debe obtener el mensaje original.
- [0.45 puntos] Con un mensaje cuyo largo no es divisible por `block_length` y en modo de operación `CBC`, se debe obtener un texto cifrado del largo correcto y un vector de inicialización `IV` al utilizar `enc`. Al utilizar `dec` con el texto cifrado y el vector de inicialización `IV` se debe obtener el mensaje original.

Esta parte tiene un total de 3 puntos. Además se va a realizar el mismo conjunto de tests pero considerando un largo de 128 bytes, 15 rondas, y la función de hash `SHA512`. Este segundo conjunto de tests también tiene 3 puntos en total, los cuales son distribuidos exactamente como en los tests anteriores. Las excepciones mencionadas en los tests pueden ser de cualquier tipo.